



PYTHON PROBLEM-SOLVING CHALLENGE

Ten themed missions across Hogwarts, the streets of New York, Mission Control, the FIFA group stage, and Kumar's tea shop — a steady climb from easy warm-ups to genuinely hard, multi-concept problems.

QUESTIONS

10 Missions

LEVEL

Easy → Hard

AUDIENCE

**Week 1 •
Intermediate**

CONCEPTS YOU'LL WIELD

Strings

Lists

Sets

Tuples

map()

filter()

Default args

*args

**kwargs

Combined

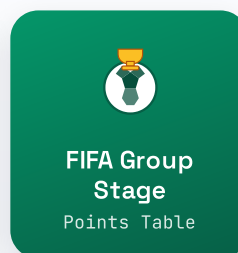
How this challenge works

Each question is a self-contained mission with a themed brief, a difficulty badge, and the exact input/output your solution must reproduce. Match the expected output precisely — spacing and formatting count.

Difficulty ladder

-  Easy
-  Easy-Medium
-  Medium
-  Medium-Hard
-  Hard

The five worlds



Read the terminals

The dark box is your **sample input**; the green box is the **expected output**. Reproduce it exactly.

Prefer the tool named

When a mission says `map()`, `filter()`, or `*args`, solve it that way — that's the muscle being trained.

Edge cases matter

Ties, invalid coordinates, empty filters — the hard questions hide their marks in the corners.

Explain your thinking

Where a prompt asks *why* (e.g. why a tuple), a one-line written answer is part of the solution.



The Sorting Hat's Verdict

Easy

Strings

The Sorting Hat receives a student's name and a mumbled string of house **signals** — letters it picked up from the student's personality. Each house is keyed by its first letter: **G**ryffindor, **H**ufflepuff, **R**avenclaw, **S**lytherin. Count how often each house letter appears (case-insensitive) and sort the student into the house with the **highest** count. On a tie, choose the house that comes first alphabetically.

SAMPLE INPUT

```
name = "Neville"  
signals = "gGhrGsgH"
```

EXPECTED OUTPUT

```
Neville, you belong in... Gryffindor! (3  
signals)
```



The Rush-Hour Report

Easy

Lists

Kumar logs cups sold every hour from 8 AM onward. Build a small report from the list: the **total** cups, the **average** per hour (rounded to 1 decimal), and every hour that beat the average — printed as the real **clock hour** (index 0 = 8 AM, index 1 = 9 AM, ...). Those “rush hours” are the ones he'll staff extra help for.

SAMPLE INPUT

```
[12, 5, 8, 20, 3, 15, 22]
```

EXPECTED OUTPUT

```
Total: 85 cups | Average: 12.1/hr  
Rush hours (above average): 8AM, 11AM,  
1PM, 2PM
```



Villain Territory Overlap

Easy-Medium

Sets

Three villains each control a set of neighborhoods. Given three lists, report: (a) the turf contested by **all three** villains, (b) neighborhoods held by **exactly one** villain — safe-ish for a solo swing-through, and (c) the total count of **distinct** neighborhoods under any villain's control.

SAMPLE INPUT

```
goblin = ["Queens", "Manhattan",
          "Brooklyn", "Bronx"]
octopus = ["Manhattan", "Brooklyn",
           "Harlem"]
vulture = ["Manhattan", "Bronx",
           "Harlem"]
```

EXPECTED OUTPUT

```
Contested by all three: {'Manhattan'}
Controlled by exactly one: {'Queens'}
Distinct neighborhoods: 5
```



Coordinate Integrity Check

Medium

Tuples

Each target is an immutable record (codename, lat, lon). Run a pre-launch integrity sweep: (a) reject any target whose latitude is outside $-90..90$ or longitude outside $-180..180$, printing it as **INVALID**, and (b) print a briefing of the valid targets sorted by latitude, north to south. In one line, justify why a **tuple** — not a list — is the right container for a locked coordinate record.

► *Fictional sci-fi mission-control simulation.*

SAMPLE INPUT

```
[("Falcon", 34.05, -118.24), ("Ghost",
 99.9, 12.0), ("Condor", 40.71, -74.00)]
```

EXPECTED OUTPUT

```
INVALID: Ghost (99.9, 12.0)
Briefing (N→S):
CONDOR → Lat: 40.71, Lon: -74.0
FALCON → Lat: 34.05, Lon: -118.24
```



Smart Billing Engine

Medium

map() + lambda

Each order is a `(item, quantity, unit_price)` tuple. Using **map()** — no explicit `for` loop over the billing math — compute the line total for each item **plus 5% GST**, rounded to 2 decimals. Then print the grand total. The entire per-line calculation must happen inside the `map()`.

SAMPLE INPUT

```
[("Masala Chai", 3, 20), ("Samosa", 2, 15), ("Green Tea", 1, 30)]
```

EXPECTED OUTPUT

```
Line totals (incl. GST): [63.0, 31.5, 31.5]
Grand total: Rs.126.00
```



Knockout Qualification

Medium

filter()

From a group's data — a list of `(team, wins, draws, losses)` tuples — compute each team's points (win = 3, draw = 1, loss = 0). Using **filter()**, return the teams that **both** scored ≥ 6 points **and** lost no more than one match. Present them sorted by points, highest first. These are the sides advancing to the knockouts.

SAMPLE INPUT

```
[("Brazil", 3, 0, 0), ("Japan", 1, 2, 0), ("Spain", 2, 0, 1), ("Ghana", 0, 1, 2)]
```

EXPECTED OUTPUT

```
Advancing to knockouts:
Brazil - 9 pts
Spain - 6 pts
```



The House Points Ledger

Medium-Hard

Default arguments

Write `award_points(house, points=10, reason="general excellence", ledger=None)` that records an award and returns the running ledger (a dict of house → total). The standard award is 10 points. The catch: the ledger must **persist and accumulate** across calls, yet you must sidestep the classic **mutable-default-argument** bug. Show, in a comment, why `ledger={}` as a default would be dangerous.

SAMPLE INPUT

```
led = award_points("Gryffindor")
led = award_points("Gryffindor", 50,
"defeating a troll", led)
led = award_points("Slytherin", 30,
ledger=led)
```

EXPECTED OUTPUT

```
Gryffindor +10 (general excellence) →
total 10
Gryffindor +50 (defeating a troll) →
total 60
Slytherin +30 (general excellence) →
total 30
Final ledger: {'Gryffindor': 60,
'Slytherin': 30}
```



Multi-Stage Launch Sequencer

Hard

*args

Write `launch(*stages, abort_threshold=5000)` where each positional argument is a stage's payload mass (kg). The sequencer must: (a) fire stages in order, printing a running cumulative mass; (b) the instant cumulative mass crosses `abort_threshold`, halt and report which stage triggered the abort; and (c) if every stage fires safely, report the total mass and stage count.

► *Fictional sci-fi mission-control simulation.*

SAMPLE INPUT

```
launch(1200, 1800, 2500, 900)
```

EXPECTED OUTPUT

```
Stage 1 armed → cumulative 1200 kg
Stage 2 armed → cumulative 3000 kg
Stage 3 armed → cumulative 5500 kg
[ABORT] at stage 3: threshold 5000 kg
exceeded.
```



Dynamic Hero Builder

Hard

*args + **kwargs

Write `create_hero(name, *powers, **stats)` combining both variadic forms. `name` is required; `*powers` collects any number of power names; `**stats` collects numeric attributes. The builder must: (a) list all powers, (b) print each stat, (c) compute an **overall rating** = the average of all numeric stat values (1 decimal), and (d) flag the hero **S-Tier** when the rating is ≥ 90 .

SAMPLE INPUT

```
create_hero("Spider-Man", "wall-crawl",  
            "spider-sense",  
            strength=85, agility=95,  
            intelligence=92)
```

EXPECTED OUTPUT

```
Hero: Spider-Man  
Powers: wall-crawl, spider-sense  
Stats:  
    strength: 85  
    agility: 95  
    intelligence: 92  
Overall rating: 90.7 → S-Tier *
```




Inventory Intelligence — Grand Finale

Hard

list · tuple · set · str · map · filter · defaults · **kwargs

The inventory is a list of tuples `(item, category, stock, price)`. Write `inventory_report(inventory, gst=0.05, **filters)` that: **(1)** prints the **set** of unique categories, alphabetically sorted; **(2)** uses **filter()** to flag **low-stock** items (`stock < 10`) that need reordering; **(3)** uses **map()** to build a `{item: price_with_gst}` view (GST rate comes from the default arg, overridable); **(4)** supports optional keyword filters via `**filters` — e.g. `category="Snack"` and/or `max_price=20` — returning only items matching **all** supplied filters; and **(5)** returns those matches as a list of item names.

SAMPLE INPUT

```
inv = [
    ("Masala Chai", "Tea", 5, 20),
    ("Green Tea", "Tea", 15, 30),
    ("Samosa", "Snack", 8, 15),
    ("Biscuit", "Snack", 25, 10),
]
inventory_report(inv, category="Snack",
max_price=15)
```

EXPECTED OUTPUT

```
Categories: ['Snack', 'Tea']
[!] Reorder soon (stock < 10): ['Masala Chai', 'Samosa']
Prices incl. GST: {'Masala Chai': 21.0,
'Green Tea': 31.5, 'Samosa': 15.75,
'Biscuit': 10.5}
Matching filters {'category': 'Snack',
'max_price': 15}: ['Samosa', 'Biscuit']
```

That's all ten.



You've crossed strings, lists, sets and tuples, bent `map()` and `filter()` to your will, and tamed default arguments, `*args` and `**kwargs`. The last three missions were meant to stretch you — revisit any that bit back.

- › Run each solution against the sample *and* one case you invent yourself.
- › If output doesn't match, diff it line by line before changing logic.
- › Refactor a working answer once — can it be shorter and still clear?
- › Stuck on the mutable-default trap in Q7? That one rewards a careful read of the docs.